LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 04 – Enkapsulasi



MUHAMMAD ‘IZZUDDIN NABIL

103132400032

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (OOP) dalam Java menggunakan *modifier* untuk mengatur tingkat akses dan sifat dari kelas, antarmuka (*interface*), *method*, maupun variabel. *Access modifier* utama meliputi *public*, *protected*, *private*, serta bawaan tanpa modifier (*default*), yang masing-masing menentukan visibilitas elemen tersebut terhadap komponen lain. Penerapan *modifier* ini sangat krusial dalam mendukung prinsip enkapsulasi (penyembunyian data), di mana praktik standarnya adalah memberikan akses *private* pada atribut agar aman dari modifikasi eksternal secara langsung, serta memberikan akses *public* pada *method* (seperti *setter* dan *getter*) yang bertindak sebagai antarmuka interaksinya.

Proses penciptaan (instansiasi) sebuah objek dari kelas membutuhkan metode khusus yang disebut *constructor*. *Constructor* memiliki aturan mutlak yaitu namanya harus persis sama dengan nama kelasnya dan tidak boleh memiliki nilai kembalian (*return value*), bahkan *void* sekalipun. Dipanggil bersamaan dengan kata kunci new, *constructor* berfungsi untuk mengalokasikan memori dan dapat digunakan untuk langsung mengisi nilai awal atribut-atribut pada objek yang baru dibuat. Sebuah kelas dapat memiliki lebih dari satu *constructor* (*overloading*), asalkan masing-masing memiliki parameter masukan yang berbeda agar program dapat membedakannya.

Untuk mengelola program dengan skala yang lebih besar, Java menggunakan *package* yang berfungsi untuk mengelompokkan kelas-kelas berdasarkan kategori tertentu. Selain membuat struktur program menjadi lebih rapi, *package* sangat penting untuk menghindari konflik penamaan jika terdapat *method* atau kelas bernama sama yang ditulis oleh pemrogram. Jika sebuah kelas perlu berinteraksi dengan kelas lain yang berada di *package* (direktori) yang berbeda, maka kelas tersebut harus memanggilnya secara eksplisit menggunakan perintah import.

B. Guided

1. Constructor

```java
package guided;

public class Manusia {
    private String nama;
    private int umur;

    // definisi constructor
    public Manusia() {
    } // constructor pertama = default tanpa parameter

    public Manusia(String a) { // constructor kedua
        nama = a;
```

```
    }

    public Manusia(String a, int b) { // constructor ketiga
        nama = a;
        umur = b;
    }

    // definisi Method
    public void setNama(String a) {
        nama = a;
    }

    public String getNama() {
        return nama;
    }

    public void setUmur(int a) {
        umur = a;
    }

    public int getUmur() {
        return umur;
    }
}
```

```
package guided;

public class DemoManusia {
    public static void main(String[] args) { // program utama
        Manusia arrMns[] = new Manusia[3]; // buat array of Object
        Manusia objMns1 = new Manusia(); // constructor pertama
        objMns1.setNama("Markonah");
        objMns1.setUmur(76);
        // constructor kedua

        Manusia objMns2 = new Manusia("Mat Conan");

        // constructor ketiga

        Manusia objMns3 = new Manusia("Bajuri", 13);
        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;
        for (int i = 0; i < 3; i++) {
```

```
            System.out.println("Nama : " + arrMns[i].getNama());
            System.out.println("Umur : " + arrMns[i].getUmur());
            System.out.println();
        }
    }
}
```

Output :

```
PS D:\Praktikum PBO>  d:; cd 'd:\Praktikum PBO'; & 'C:\Program
onMessages' '-cp' 'C:\Users\Lenovo\AppData\Roaming\Code\User\w
t_ws\jdt.ls-java-project\bin' 'guided.DemoManusia'
Nama : Markonah
Umur : 76

Nama : Mat Conan
Umur : 0

Nama : Bajuri
Umur : 13
```

2. Package

```
package hargapulsa;

public class hargapulsa {
    public void info() {
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}
```

```
package hargatoken;

public class hargatoken {
    public void info() {
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
```

```
        System.out.println("Rp. 500000");
    }
}
```

```
package main;

import hargatoken.hargatoken;
import hargapulsa.hargapulsa;

public class main {
    public static void main(String[] args) {
        hargatoken objectToken = new hargatoken();
        objectToken.info();
        hargapulsa objectPulsa = new hargapulsa();
        objectPulsa.info();
    }
}
```

Output :

```
PS D:\Praktikum PBO>  & 'C:\Program Files\Java\jdk-17\bin\java.exe' '-
\Lenovo\AppData\Roaming\Code\User\workspaceStorage\0499d1d5b3c2a39c285
\bin' 'main.main'
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
```

C. Unguided

1. Soal Unguided 1

   Sistem Pencatatan Dataset

   Kamu sedang membangun program kecil untuk mencatat dan memeriksa kualitas dataset sebelum dianalisis, seperti tahap awal pengecekan missing value. Setiap dataset punya nama, jumlah baris, jumlah kolom, dan jumlah sel yang kosong (missing).

   Petunjuk Umum

   Buat **3 package** dengan struktur berikut:

   Source Packages

   ├── model

```
│   └── Dataset.java
├── laporan
│   └── LaporanDataset.java
└── main
    └── Main.java
```

1. Class Dataset (package model)

Atribut (implementasikan enkapsulasi):

Atribut	Tipe	Keterangan
nama	String	Nama dataset
jumlahBaris	int	Banyak baris (observasi)
jumlahKolom	int	Banyak kolom (fitur)
jumlahMissing	int	Banyak sel yang kosong

Ketentuan tambahan:

- Buat **konstanta** BATAS_MISSING bernilai 5.0 (persen). Dataset dengan missing diatas batas ini dianggap perlu dibersihkan.
- Buat **variabel milik class** yang menghitung total objek Dataset yang telah dibuat, beserta **method** untuk ambil nilainya tanpa perlu membuat objek.

Constructor (ada 3, overloading):

1. Tanpa parameter
2. Hanya parameter nama
3. Lengkap: nama, jumlahBaris, jumlahKolom, jumlahMissing

Setiap constructor harus menambah hitungan total dataset.

Method:

- Getter dan setter untuk semua atribut.
- Pada setter jumlahBaris, jumlahKolom, dan jumlahMissing, buat agar nilai tidak berubah.
- getPersentaseMissing() mengembalikan double: persentase sel kosong dibanding total sel (baris × kolom). Jika total sel = 0, kembalikan 0.
- perluDibersihkan() mengembalikan boolean: true jika persentase missing melebihi BATAS_MISSING.

2. Class LaporanDataset (package laporan)

Buat method cetak yang menerima satu objek Dataset dan menampilkan laporannya dengan format:

=== Laporan Dataset ===
Nama : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing : 866 sel (8.10%)
Status : Perlu dibersihkan
Status berisi Perlu dibersihkan atau Bersih sesuai hasil dari perluDibersihkan().

3. Class Main (package main)

Buat tiga objek, masing-masing dengan constructor berbeda:

Objek	**Cara pembuatan**	**Data**
1	Constructor tanpa parameter, lalu isi memakai setter	Titanic, 891 baris, 12 kolom, 866 missing
2	Constructor 1 parameter	Wine Quality
3	Constructor lengkap	Iris, 150 baris, 5 kolom, 0 missing

Simpan ketiganya dalam **array**, cetak laporannya dengan perulangan, lalu tampilkan di akhir:
Total dataset dibuat : 3

Jawaban :
Dataset

```java
package model;

public class Dataset {
    private String nama;
    private int jumlahBaris;
    private int jumlahKolom;
    private int jumlahMissing;
    private static final double BATAS_MISSING = 5.0;
    private static int totalDataset = 0;

    public Dataset(){
        totalDataset++;
```

```
    }

    public Dataset(String nama){
        this.nama = nama;
        totalDataset++;
    }

    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing){
        this.nama = nama;
        this.jumlahBaris = jumlahBaris >= 0 ? jumlahBaris : 0;
        this.jumlahKolom = jumlahKolom >= 0 ? jumlahKolom : 0;
        this.jumlahMissing = jumlahMissing >= 0 ? jumlahMissing : 0;
        totalDataset++;
    }

    public static int getTotalDataset(){
        return totalDataset;
    }

    public String getNama(){
        return nama;
    }

    public void setNama(String nama){
        this.nama = nama;
    }

    public int getJumlahBaris(){
        return jumlahBaris;
    }

    public void setJumlahBaris(int jumlahBaris){
        if (jumlahBaris >= 0) {
            this.jumlahBaris = jumlahBaris;
        }
    }

    public int getJumlahKolom(){
        return jumlahKolom;
    }

    public void setJumlahKolom(int jumlahKolom){
        if (jumlahKolom >= 0) {
            this.jumlahKolom = jumlahKolom;
```

```
        }
    }

    public int getJumlahMissing(){
        return jumlahMissing;
    }

    public void setJumlahMissing(int jumlahMissing){
        if (jumlahMissing >= 0) {
            this.jumlahMissing = jumlahMissing;
        }
    }

    public double getPersentaseMissing(){
        int totalSel = jumlahBaris * jumlahKolom;
        if (totalSel == 0) {
            return 0.0;
        }
        return ((double) jumlahMissing / totalSel) * 100;
    }

    public boolean perluDibersihkan(){
        return getPersentaseMissing() > BATAS_MISSING;
    }
}
```

Laporan dataset

```
package laporan;

import java.util.Locale;
import model.Dataset;

public class LaporanDataset {
    public void cetak(Dataset d){
        System.out.println("=== Laporan Dataset ===");
        String namaTampil = (d.getNama() != null) ? d.getNama() :
"Belum ada nama";
        System.out.println("Nama         : " + namaTampil);
        System.out.println("Jumlah Baris : " + d.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + d.getJumlahKolom());
        System.out.printf(Locale.US, "Missing      : %d sel
(%.2f%%)%n", d.getJumlahMissing(), d.getPersentaseMissing());
        String status = d.perluDibersihkan() ? "Perlu dibersihkan" :
"Bersih";
```

```
        System.out.println("Status       : " + status);
        System.out.println();
    }
}
```

main

```
package main;

import model.Dataset;
import laporan.LaporanDataset;

public class Main {
    public static void main(String[] args) {
        Dataset ds1 = new Dataset();
        ds1.setNama("Titanic");
        ds1.setJumlahBaris(891);
        ds1.setJumlahKolom(12);
        ds1.setJumlahMissing(866);
        Dataset ds2 = new Dataset("Wine Quality");
        Dataset ds3 = new Dataset("Iris", 150, 5, 0);
        Dataset[] daftarDataset = { ds1, ds2, ds3 };

        LaporanDataset pencetak = new LaporanDataset();
        for (Dataset ds : daftarDataset) {
            pencetak.cetak(ds);
        }
        System.out.println("Total dataset dibuat : " +
Dataset.getTotalDataset());
    }
}
```

Output :

```
PS D:\Praktikum PBO>  & 'C:\Program Files\Java\jdk-17\bin\java.exe'
\Lenovo\AppData\Roaming\Code\User\workspaceStorage\0499d1d5b3c2a39c2
\bin' 'main.Main'
=== Laporan Dataset ===
Nama         : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing      : 866 sel (8.10%)
Status       : Perlu dibersihkan

=== Laporan Dataset ===
Nama         : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing      : 0 sel (0.00%)
Status       : Bersih

=== Laporan Dataset ===
Nama         : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing      : 0 sel (0.00%)
Status       : Bersih

Total dataset dibuat : 3
```

Penjelasan :

Program pada tugas unguided merupakan implementasi sistem pencatatan dan evaluasi kualitas dataset sederhana dengan menerapkan prinsip utama Enkapsulasi. Konstruktor Overloading, Static Member, serta Arsitektur Multi-Package.

Program dibagi ke dalam tiga package berbeda untuk menerapkan prinsip pemisahan tanggung jawab (Separation of Concerns):

1. Package model (Dataset.java):
   - Berfungsi sebagai representasi entitas data (data model).
   - Seluruh atribut (nama, jumlahBaris, jumlahKolom, jumlahMissing) dideklarasikan dengan access modifier private untuk mencegah manipulasi data secara langsung dari luar class.
   - Akses data dikendalikan melalui method getter dan setter. Pada method setter diberi validasi logika (hanya menerima nilai ≥ 0) sehingga integritas data tetap terjaga.
   - Memiliki konstanta private static final double BATAS_MISSING = 5.0 sebagai acuan ambang batas kualitas dataset.

- Memiliki variabel kelas private static int totalDataset dan method public static int getTotalDataset() untuk melacak akumulasi objek dataset yang telah dibuat tanpa bergantung pada instansiasi objek tertentu.
- Menerapkan Constructor Overloading dengan 3 variasi parameter (tanpa parameter, 1 parameter nama, dan parameter lengkap). Setiap kali konstruktor dipanggil, nilai totalDataset akan bertambah secara otomatis (totalDataset++).
- Menyediakan fungsi komputasi data: getPersentaseMissing() untuk menghitung rasio missing value terhadap total sel data, serta perluDibersihkan() yang mengembalikan nilai boolean berdasarkan ambang batas.

2. Package laporan (LaporanDataset.java):
   - Berfungsi khusus menangani presentasi dan pencetakan data ke layar (reporting layer).
   - Method cetak(Dataset d) menerima referensi objek Dataset, membaca atributnya melalui getter, dan menampilkan ringkasan data dengan format terstruktur serta presisi angka desimal (printf dua angka di belakang koma).
3. Package main (Main.java):
   - Merupakan titik masuk (entry point) eksekusi program.
   - Menguji ketiga konstruktor berbeda untuk tiga objek dataset (Titanic, Wine Quality, dan Iris).
   - Menyimpan seluruh objek ke dalam array (Dataset[]), melakukan iterasi pencetakan laporan secara otomatis menggunakan perulangan for-each, dan menampilkan total dataset yang dibuat melalui pemanggilan method statis Dataset.getTotalDataset().

D. Kesimpulan

Berdasarkan praktikum Modul 4 (Enkapsulasi), dapat diambil beberapa kesimpulan penting:

1. Pentingnya Enkapsulasi (Data Hiding): Melalui latihan guided dan unguided, enkapsulasi terbukti efektif melindungi integritas data suatu objek dengan cara menyembunyikan atribut menggunakan modifier private dan membatasi akses melalui getter dan setter. Dengan mekanisme ini, kita dapat menaruh aturan/validasi data pada setter agar nilai atribut tidak diisi secara sembarangan dari luar kelas.
2. Fleksibilitas Objek Melalui Constructor Overloading: Pada latihan guided (Manusia) maupun unguided (Dataset), konsep constructor overloading memberikan fleksibilitas tinggi saat instansiasi objek. Objek dapat diinisialisasi dalam berbagai kondisi data (nilai awal kosong/default, terisi sebagian, maupun terisi penuh) sesuai kebutuhan saat runtime.
3. Pengorganisasian Kode dengan Package: Pada latihan guided (package pulsa dan token) serta unguided, penggunaan package memungkinkan kode program tersusun rapi secara modular sesuai fungsinya. Pemisahan kelas ke dalam package berbeda mempermudah

pemeliharaan kode (maintainability), menghindari konflik nama kelas (namespace collision), serta mendukung arsitektur aplikasi berbasis modul.

4. Pemanfaatan Keyword static: Atribut dan method berlabel static membuktikan bahwa data dapat dimiliki secara bersama oleh seluruh instansi suatu kelas (class-level). Hal ini sangat berguna untuk keperluan pencatatan global (seperti counter total objek yang tercipta) tanpa harus melekat pada salah satu objek spesifik.